PROG7311 PORTFOLIO OF EVIDENCE

Part 1

Name	Sujendran Reddy
Student Number	10457761
Module	PROG7311
Lecturer	Mortda Adam
Date of Submission	2026/03/27

I hereby declare that I did not plagiarise the content of this assignment and that this is my own work. I also declare that no use of GenAI was used in the creation of this submission (unless explicitly permitted and declared), excluding grammar and spell checkers. I consent to the lecturer assessing our work using an AI detection tool to ensure the integrity of our work.

Assignment submitted via SafeAssign: ☒ (Tick the Box)

Contents

1.1 EXECUTIVE SUMMARY 3

1.2 INTRODUCTION 3

1.3 ENTERPISE FRAMEWORK STRATEGY 4

1.4 FRAMEWORK APPLICATION 4

1.5 PATTERN SELECTION & JUSTIFICATION 5

1.6 DESIGN PATTERNS LOGIC & STRUCTURE 7

1.7 UML MODELING 9

1.8 NON-FUNCTIONAL REQUIREMENTS & SCALABILITY 10

1.9 REFERENCE LIST 12

1.1 EXECUTIVE SUMMARY

TechMove's current system relies on an outdated legacy system, which has led to inefficiencies, inconsistencies, and operational risks. This report proposes the Global Logistics Management System, an enterprise-grade platform designed to streamline logistics, centralize contract management, and support integration with external systems. The proposed architecture leverages frameworks and design patterns to ensure scalability, reliability, and maintainability. Key non-functional requirements, such as availability, interoperability, and scalability, are addressed to support future growth. This solution provides a resilient, scalable foundation capable of handling increased demand following business expansion.

1.2 INTRODUCTION

TechMove Logistics currently relies on a fragmented legacy system, which includes spreadsheets, emails, and other manual processes. These methods often result in lost invoices, data inconsistency, and operational inefficiencies. To combat these challenges, the Global Logistics Management System (GLMS) is proposed. It is designed to streamline service requests, support financial integration, and centralize contract management. This report presents a scalable and resilient architectural vision for the GLMS. The report outlines the enterprise's selected framework, key design patterns, and non-functional requirements. Demonstrating how the system will support future growth, integration, and high-volume workloads.

1.3 ENTERPISE FRAMEWORK STRATEGY

The Zachman Framework aims to account for and harmonize stakeholders and developers with the specific issue being addressed (SAP SE, 2026). The framework utilizes a matrix made up of 36 cells, each of which focuses on a unique aspect of the business. The cells are organized into columns and rows (ERP Informer, 2022). The 6 columns include: What (data), How (Function), Where (Network), Who (People), When (Time), and Why motivation. The 6 rows include: Planner, Owner, Designer, Builder, Subcontractor, and Functioning Enterprise (Umbrex, 2026). The Zachman Framework ensures comprehensive coverage, improved alignment, enhanced communication, streamlined decision-making, scalability, flexibility, operational efficiency, consistency, and standardization (Ardoq, 2024). This adds great value to the GLMS, as the strategy supports the organization of processes, data, and infrastructure. Furthermore, the Zachman Framework enables clear communication, allowing stakeholders to provide a solid foundation for scalability, integration, and future expansion.

1.4 FRAMEWORK APPLICATION

Perspective	What (Data)	How (Function)	Where (Network)
Planner (Scope / Contextual)	- Contracts (SLA, status) - Clients (global customers) - Service Requests - Invoices & payments	- Contract management - Service request processing- Invoice generation - Validation of active contracts	- Global operations (multi-region) - Cloud-based system - Access by logistics managers and partners
Designer (System Model / Logical)	- Structured contract entities- Client and request relationships - Normalised database schema - Data formats (JSON/XML)	- API-based processing (REST services) - Business logic for contract validation- Automated workflows (request approval) - Integration with external APIs	- Distributed cloud architecture (AWS/Azure) - Microservices deployment- API Gateway for routing - Load-balanced servers

1.5 PATTERN SELECTION & JUSTIFICATION

The GLMS calls for a scalable and loosely coupled architectural design in order to combat the issues of the current legacy system. These include data fragmentation, inefficient communication, and the lack of validation. To solve these challenges, three GoF design patterns have been selected: Factory, Strategy, and Observer (Lamrani, 2026). These Patterns have been selected to address object creation, validation logic, and real-time notifications within the system.

Factory Pattern

TechMove Logistics handles multiple contract types. The current outdated legacy system depends on manual creation, which often leads to inconsistent information, a lack of standards, and no validation rules. The Factory Pattern provides abstraction and encapsulates object creation into a single component (Coding Explanations, 2023). It will simplify contract creation of different types through a common interface (e.g., IContract) and eradicate the tight coupling between contract creation and business logic (Cloud Technology, 2025). It allows for validation rules to be added during creation. The pattern will ensure consistent standardized contract creation and support scalability, allowing new contract types to be added without affecting or altering existing code (Kumar, 2024). All stored contracts within the Contract Management Hub will be created in a consistent and valid format and will support both accurate status tracking and data integrity. This directly addresses TechMove's current validation and decoupling issues by separating object creation and enforcing validation rules.

Strategy Pattern

Different contracts will require a different set of validation rules and logic. Hardcoding these rules would likely result in reduced flexibility, security risks, scalability issues, and difficult maintenance (JuniorToExpert, 2025). The Strategy Pattern encapsulates validation logic into separate strategies, allowing the system to select the correct validation logic based on the type of contract (Strategy Pattern, 2018). This Pattern carries many benefits, such as enhanced flexibility (change strategies at runtime), better maintainability, and improved testing (less risk of introducing bugs because strategies are encapsulated into separate classes) (Garcia, 2025). The Pattern can also support financial integration by encapsulating different currency conversion algorithms taken from external APIs (GeeksforGeeks, 2016b). Furthermore, the system will allow Logistics managers to raise service requests. These requests will be validated to ensure they are raised against valid, active contracts. The System Pattern aligns well with the needs of the GLMS, specifically for validation, standardisation and financial integration.

Observer Pattern

TechMove Logistics requires various real-time notifications (status changes, request updates, and invoice generation) but currently relies on manual communication, such as emails and phone calls, which will likely result in delays, errors, and inefficiencies. The Observer Pattern implements a system where when the state of one object changes, all of the "subscribers" of that object are automatically updated (GeeksforGeeks, 2016a). This Pattern will implement automated notifications such as status updates, eliminating the need for manual communication and ensuring real-time visibility across the entire system. It also comes with benefits such as loose coupling (does not need to know about the

subject, making it easy to remove or add), flexibility (can be used in various applications), and maintainability (separates concerns) (Glushenkov, 2023).

As a collective, these three design patterns directly address TechMove's current issues and align with the GLMS's functional requirements. They will ensure strict validation, real-time notifications, enforce standardization, and support scalability.

1.6 DESIGN PATTERNS LOGIC & STRUCTURE

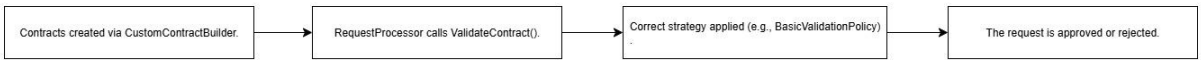
Factory Pattern Logic

The Factory Pattern defines the IContractBuilder interface to create contracts. The CustomContractBuilder implements the creation logic for each contract type. IShippingContract is the base interface for all contracts (StandardShippingContract, PremiumShippingContract, and GlobalShippingContract). The Factory Pattern decouples contract creation from business logic. ContractManager uses IContractBuilder to create contracts. The IContractBuilder method is injected into ContractManager constructor (e.g., IContractBuilder builder). The builder will apply default validation rules during contract creation. This pattern standardizes contract creation in the Contract Management Hub and supports the status tracking system.

Strategy Pattern Logic

The IValidationPolicy defines the ValidateContract(contract) method. The concrete strategies will include BasicValidationPolicy, PremiumValidationPolicy, and ActiveContractPolicy (ensures requests are only for active contracts). The RequestProcessor method holds a reference to IValidationPolicy and can switch strategies at runtime based on the contract type being created. RequestProcessor will receive the validation policy through a constructor (e.g., RequestProcessor(IValidationPolicy policy).

The flow of Validation:



The pattern ensures service requests are only for active contracts, supports financial API integration, and enforces validation throughout the contract creation methods.

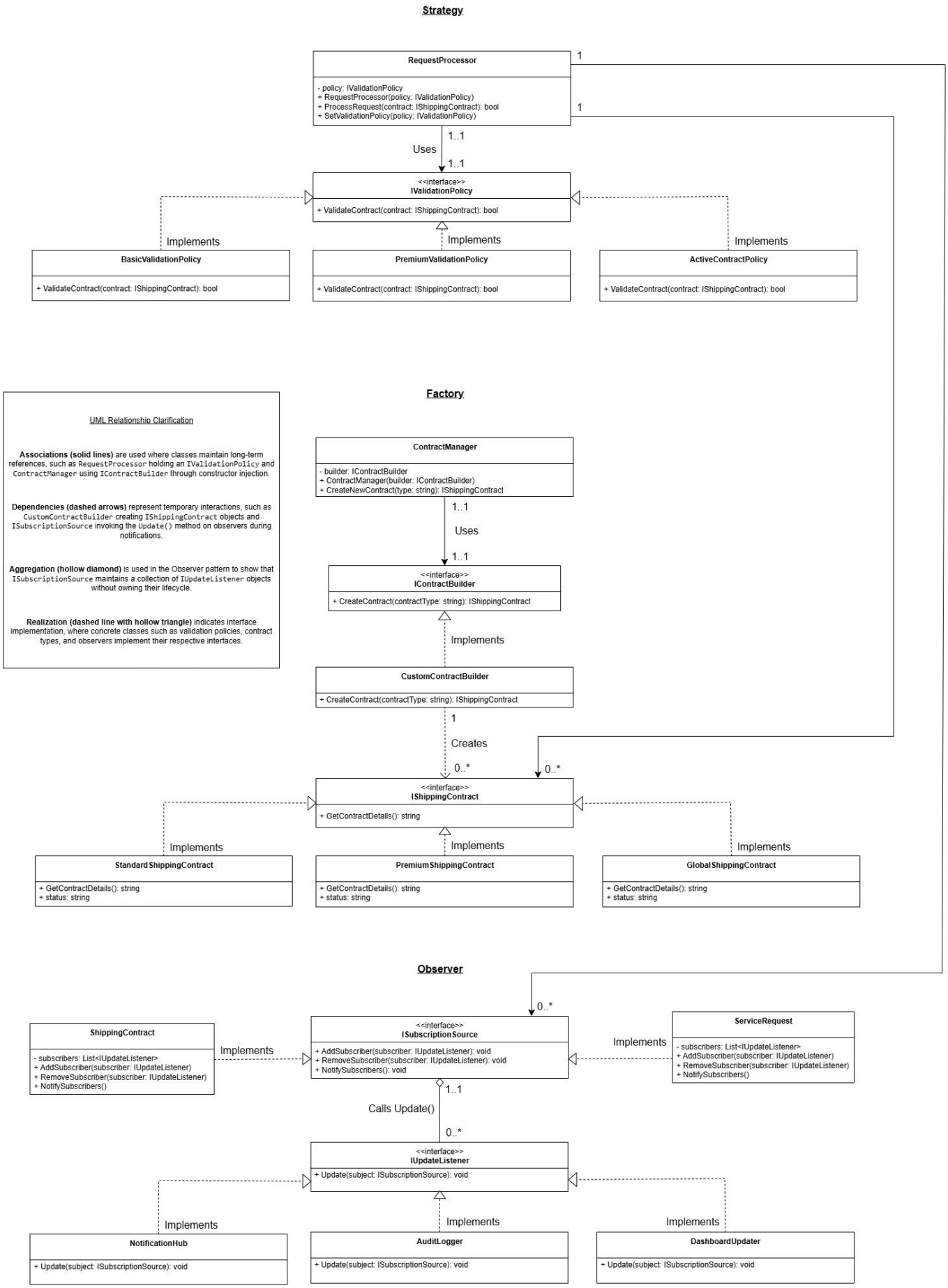
Observer Pattern Logic

ISubscriptionSource defines multiple methods: AddSubscriber(IUpdateListener), RemoveSubscriber(IUpdateListener), and NotifySubscribers(). The IUpdateListener defines the Update() method. The concrete observers include NotificationHub (sends emails, SMS), AuditLogger (logs system events), and the DashboardUpdater (updates the UI automatically). To demonstrate the relationship: ShippingContract and ServiceRequest are subjects, and the observers subscribe to them to get updates. Events trigger the updates. If a service request changes state, the NotifySubscribers() method will be called, and all observers will be updated automatically via the Update() method. Observers are injected dynamically. The pattern enforces real-time notifications for status change, supports automated contract status tracking, and reduces manual communication discrepancies.

Complete Flow

The three design patterns support decoupling by separating creation, validation, and notifications. They support scalability by allowing developers to easily integrate new contract types, validation policies, or observers, without altering existing code, and maintain business rules and data integrity.

1.7 UML MODELING



1.8 NON-FUNCTIONAL REQUIREMENTS & SCALABILITY

Non-functional requirements define how a system performs rather than how it functions; they focus on factors such as availability, interoperability, and scalability (Weingartz, 2025). Availability is defined as the system's ability to be operational and accessible to users when needed. Regarding the GLMS, the system must run 24/7, and contracts must be created, accessed, and updated in real-time. System downtime would lead to major business disruptions and delays in logistics operations. To ensure the GLMS maintains strong availability, it will use high availability architecture (High-Availability, 2026). This involves implementing redundant components (such as multiple servers), failover mechanisms (switching servers if one fails), replication (duplicating data/services), disaster recovery (procedures to recover from failure), and load balancing (distributing incoming traffic across multiple servers) (Mitchell, 2024). Cloud infrastructures such as Azure and AWS would greatly benefit the GLMS's availability by increasing the system's failure detection (Atatus, 2025). Utilizing high availability architecture, single point failure will be avoided, global operations will be supported and continuous contract processing will be ensured (Troutman, 2021).

Interoperability is defined as the ability for systems to share functionalities and exchange information with little to no user involvement (IBM, 2025). The GLMS requires integration with various systems, external logistics providers, warehouse systems, shipment tracking systems, and payment systems. To enhance the interoperability of the system, RESTful APIs will be integrated. APIs ensure seamless data exchange and synchronization (Kate, 2025). The system will utilize standard data formats such as JSON and XML, ensuring data can be understood across various systems (Orlova, Rhodes, and Warner, 2024). API gateways will be used as a central access point for all services (GeeksforGeeks, 2023). Middleware will enable communication between the various systems, further enhancing its interoperability (Varad, 2025). Increasing the system's interoperability enables easy integration, supports scalability, and reduces tight coupling.

"If TechMove acquires a competitor and adds 50,000 new contracts overnight, how will the systems architecture handle the load?" The system will handle sudden increases in load through scalability. Scalability is defined as the system's ability to efficiently and effectively handle an increase in users, data, and workloads (GeeksforGeeks, 2022). There are primarily 2 types of scaling, horizontal and vertical (Bouguezzi, 2022). Horizontal scaling involves increasing machines or nodes in the existing infrastructure in order to distribute workloads across multiple servers (Amnic, 2025). Vertical scaling, according to Amnic (2025), "involves enhancing the resources of existing servers by adding more CPU, RAM, or storage. Horizontal scaling better suits the GLMS's needs, as it increases capacity, performance, and fault tolerance (GeeksforGeeks, 2021a).

The GLMS will scale horizontally by using various methods. Resources will be added to the servers, increasing memory, CPU, disk space, or simply adding more servers (El Bokhary, 2023). Load balancing will be used to distribute network traffic across resources, preventing bottlenecks, predicting traffic, and adding redundancy (Amazon Web Services, 2024). Cache data will be in the system due to frequently accessed contracts, customer data, and much more. Redis caching will be implemented to store frequently accessed data in memory, reducing application response time which will make the

application faster (GeeksforGeeks, 2023a). To further handle the contract surge, database sharing, where data is split across multiple instances, will also be implemented to reduce the impact of large amounts of data on a singular database (GeeksforGeeks, 2021b). To handle decoupling between senders and receivers by acting as buffers, asynchronous message queues will be used (GeeksforGeeks, 2023b). This will allow the system to operate when certain components are delayed or unavailable (GeeksforGeeks, 2023b). Lastly, to further enhance the system's scalability, auto-scaling will be used, allowing the system to increase and decrease resources, efficiently handling workload spikes (GeeksforGeeks, 2024). These combined efforts will ensure the GLMS will efficiently handle a sudden increase in workload without performance degradation. These strategies will ensure it can dynamically adjust to increased demands, maintain fast response time, prevent bottlenecks, and avoid downtime.

In conclusion, availability ensures continuous system operation, interoperability allows for seamless integration, and scalability ensures the GLMS can handle increased workloads. Together, these non-functional requirements ensure the system is reliable, flexible, and capable of supporting business expansion.

Total Word Count: 1496

Excluding headings, diagrams, tables, and references.

1.9 REFERENCE LIST

Amazon Web Services, 2024. *What is Load Balancing? Load Balancing Explained - AWS*. [online] Amazon Web Services, Inc. Available at: <https://aws.amazon.com/what-is/load-balancing/> [Accessed 27 March 2026].

Ardoq, 2024. *What is the Zachman Framework? A Definitive Guide to this EA Standard*. [online] Ardoq.com. Available at: <https://www.ardoq.com/knowledge-hub/zachman-framework> [Accessed 27 March 2026].

Atatus, 2025. *Best Cloud Monitoring Platforms 2025 - AWS, Azure, GCP | Atatus*. [online] Atatus.com. Available at: <https://www.atatus.com/guides/best-cloud-monitoring-platforms/> [Accessed 26 March 2026].

Bouguezzi, S., 2022. *Differences Between Scaling Horizontally and Vertically | Baeldung on Computer Science*. [online] www.baeldung.com. Available at: <https://www.baeldung.com/cs/scaling-horizontally-vertically> [Accessed 27 March 2026].

Cloud Technology, 2025. *CloudComputingTechnology*. [online] Masarbi.com. Available at: <https://masarbi.com/post/what-is-a-factory-pattern-in-software-engineering/> [Accessed 25 March 2026].

Coding Explanations, 2023. *Coding Explorations*. [online] Coding Explorations. Available at: <https://www.codingexplorations.com/blog/exploring-the-importance-and-benefits-of-the-factory-pattern-in-software-engineering> [Accessed 25 March 2026].

El Bokhary, N.A., 2023. *What are the most effective ways to handle increased web traffic?* [online] www.linkedin.com. Available at: <https://www.linkedin.com/advice/3/what-most-effective-ways-handle-increased-ji0ne> [Accessed 27 March 2026].

ERP Informer, 2022. *Zachman Framework (Example, Benefits, and Certification)*. [online] ERP Information. Available at: <https://www.erp-information.com/zachman-framework#How_Does_It_Work> [Accessed 27 March 2026].

Garcia, I., 2025. *Mastering Strategy Patterns: Build Scalable and Flexible Architectures - Buildonline*. [online] Buildonline.io. Available at: <https://www.buildonline.io/blog/mastering-strategy-patterns-build-scalable-and-flexible-architectures> [Accessed 25 March 2026].

GeeksforGeeks, 2016a. *Observer Design Pattern*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/observer-pattern-set-1-introduction/> [Accessed 25 March 2026].

GeeksforGeeks, 2016b. *Strategy Design Pattern*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/strategy-pattern-set-1/> [Accessed 25 March 2026].

GeeksforGeeks, 2020. *Functional vs. Non Functional Requirements*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/software-engineering/functional-vs-non-functional-requirements/> [Accessed 26 March 2026].

GeeksforGeeks, 2021. *Horizontal and Vertical Scaling | System Design*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/system-design-horizontal-and-vertical-scaling/> [Accessed 27 March 2026].

GeeksforGeeks, 2022. *What is Scalability and How to achieve it?* [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/what-is-scalability/> [Accessed 27 March 2026].

GeeksforGeeks, 2023. *What is API Gateway | System Design?* [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/what-is-api-gateway-system-design/> [Accessed 26 March 2026].

GeeksforGeeks, 2024. *What is Auto Scaling?* [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/what-is-auto-scaling/> [Accessed 27 March 2026].

Glushenkov, A., 2023. *Notifying the Masses: The Benefits and Challenges of the Observer Pattern in Software Development*. [online] Medium. Available at: <https://medium.com/@alexglushenkov/notifying-the-masses-the-benefits-and-challenges-of-the-observer-pattern-in-software-development-5decd1c008e3> [Accessed 25 March 2026].

High-Availability, 2026. *High Availability Architecture | HA Systems*. [online] High-availability.com. Available at: <https://www.high-availability.com/articles/architecture/> [Accessed 26 March 2026].

IBM, 2025. *Interoperability*. [online] Ibm.com. Available at: <https://www.ibm.com/think/topics/interoperability> [Accessed 26 March 2026].

JuniorToExpert, 2025. *What is Hardcode? Advantages and Disadvantages*. [online] Junior to Expert. Available at: <https://juniortoexpert.com/en/what-is-hardcode/> [Accessed 25 March 2026].

Kate, 2025. *How Does Api Integration Improve System Interoperability?* [online] Technology & Finance Blog. Available at: <https://devtalk.digitalpress.blog/how-does-api-integration-improve-system-interoperability/> [Accessed 26 March 2026].

Kumar, S., 2024. *Design Patterns and Advantages of Different Design Patterns*. [online] C-sharpcorner.com. Available at: <https://www.c-sharpcorner.com/article/design-patterns-and-advantages-of-different-design-patterns/> [Accessed 25 March 2026].

Lamrani, A., 2026. *Mastering the GoF Design Patterns: Explained with Modern Examples*. [online] Themorningdev.com. Available at: <https://themorningdev.com/software-architecture-design/gang-of-four-design-patterns/> [Accessed 25 March 2026].

Mitchell, T., 2024. *High Availability Architecture: Requirements & Best Practices*. [online] The Couchbase Blog. Available at: <https://www.couchbase.com/blog/high-availability-architecture/> [Accessed 26 March 2026].

Orlova, A., Rhodes, H. and Warner, D., 2024. *Standardizing Data and HIM Practices for Interoperability HIT Standards, Standardization, and Systems Interoperability-An Overview*. [online] Available at: <https://journal.ahima.org/Portals/0/archives/AHIMA%20files/Standardizing%20Data%20and%20HIM%20Practices%20for%20Interoperability.pdf> [Accessed 26 March 2026].

SAP SE, 2026. *A Definitive Guide to Zachman Framework*. [online] SAP LeanIX. Available at: <https://www.leanix.net/en/wiki/ea/zachman-framework?crm_code=CRM-XG26-LIX-449105&utm_term=zachman%20framework&utm_source=bing-ads&utm_medium=ppc&utm_campaign=EMEA-MEA-SOUTH_Enterprise-Architecture_EA_Search_ENG_B&hsa_acc=9751618594&hsa_cam=590194741&hsa_grp=1150090881405628&hsa_kw=zachman%20framework&hsa_mt=e&hsa_net=bing&hsa_src=o&hsa_tgt=kwd-71881374390370:loc-168&hsa_ad=&hsa_ver=3&msclkid=b81b797605f3172f1a0b10b067040876&utm_content=MEA-SOUTH_enterprise-architecture-zachman-framework> [Accessed 27 March 2026].

Strategy Pattern, 2018. *The Strategy Pattern*. [online] Pmi.org. Available at: <https://www.pmi.org/disciplined-agile/the-design-patterns-repository/the-strategy-pattern> [Accessed 25 March 2026].

Troutman, M., 2021. *High Availability (HA): What Are the Benefits?» LINBIT*. [online] LINBIT. Available at: <https://linbit.com/blog/high-availability-benefits/> [Accessed 26 March 2026].

Umbrex, 2026. *Zachman Framework*. [online] Independent Management Consultants. Available at: <https://umbrex.com/resources/frameworks/project-management-frameworks/zachman-framework/> [Accessed 27 March 2026].

Varad, 2025. *Enhancing Interoperability with APIs and Middleware - Softat*. [online] Softat. Available at: <https://softat.co.in/use-of-apis-and-middleware-for-interoperability/> [Accessed 26 March 2026].

Weingartz, R., 2025. *Non-Functional Requirements: A Comprehensive Guide*. [online] aqua cloud - best software for testing. Available at: <https://aqua-cloud.io/understanding-non-functional-requirements-software-engineering/> [Accessed 26 March 2026].